

VIDEO 1 - Demo Automation Testing đa trình duyệt

Người trình bày	Nguyễn Tấn Thắng - 23127259
Ngôn ngữ	Tiếng Việt
Thời lượng mục tiêu	8-10 phút, tuyệt đối không dưới 5 phút
Feature chạy live	FR-07 - Shopping Cart
Evidence tác giả	Face-cam hoặc terminal hiển thị <code>whoami</code> và <code>hostname</code>
Chế độ YouTube	Unlisted

Đề yêu cầu video chính được thuyết minh bằng **tiếng Việt**. Không cần nói tiếng Anh; tên file, câu lệnh và thuật ngữ kỹ thuật có thể giữ nguyên tiếng Anh.

PHẦN A - CHUẨN BỊ TRƯỚC KHI BẮM REC

A1. Chuẩn bị màn hình

Mở sẵn ba cửa sổ:

1. **VS Code/Codex:** repository `HW04-TESTING`.
2. **Terminal:** đang ở thư mục gốc repository.
3. **Chrome:** mở sẵn các tab:
 - `https://github.com/hungtmh/HW04-TESTING`
 - `https://github.com/hungtmh/HW04-TESTING/issues/25`
 - `https://github.com/hungtmh/HW04-TESTING/pull/36`

Trong VS Code mở sẵn:

- `23127259/README.md`
- `tests/fr07-cart.spec.js`
- `tests/pages/CartPage.js`
- `tests/data/fr07-cart-products.csv`
- `eshop-sut/README.md` tại phần FR-07

- `23127259/bug-report/BUG_REPORT.md` tại `BUG-10/BUG-11`

A2. Cài đặt trước khi quay

Chạy trước, không cần quay đoạn download:

```
cd /Volumes/Thang/HW04/HW04-TESTING
npm install
npx playwright install chromium firefox webkit
```

Playwright config sẽ tự khởi động Backend, Web và Admin. Không cần mở ba terminal server riêng.

A3. Kiểm tra nhanh trước khi quay

```
git branch --show-current
git status --short
node --check tests/fr07-cart.spec.js
```

Không dùng `npm run test` cho toàn repository vì sẽ chạy cả feature của thành viên khác. Video này dùng đúng:

```
npm run test:multibrowser:fr07
```

A4. Checklist trước REC

- ☐ Micro rõ, không có tiếng vọng.
- ☐ Face-cam bật hoặc sẽ chạy cả `whoami` và `hostname`.
- ☐ Font terminal/VS Code đủ lớn.
- ☐ Tắt notification và ẩn thông tin nhạy cảm.
- ☐ Chrome đang đăng nhập tài khoản GitHub `thangak18`.
- ☐ Không mở file có token, cookie hoặc credential hệ thống.
- ☐ Có ít nhất 10 phút trống để quay một mạch.

PHẦN B - KỊCH BẢN QUAY VÀ LỜI THOẠI

Mục 1 - Mở đầu và chứng minh tác giả (0:00-0:50)

Thao tác

Mở terminal, phóng to và chạy:

```
whoami  
hostname  
pwd
```

Lời thoại đọc gần như nguyên văn

Xin chào thầy. Tôi là sinh viên Nguyễn Tấn Thắng, mã số sinh viên 23127259. Đây là video demo bài HW04 Automation Testing của tôi. Tôi trình bày bằng tiếng Việt theo yêu cầu đề bài. Trên màn hình là kết quả lệnh whoami, hostname và đường dẫn repository để chứng minh môi trường tôi đang sử dụng. Hệ thống kiểm thử là EShop gồm Backend Node.js, Frontend Web và Frontend Admin.

Mục 2 - Phạm vi bài làm và số liệu tổng quan (0:50-1:50)

Thao tác

Chuyển sang `23127259/README.md`, chỉ vào bảng thông tin và Test Summary Report.

Lời thoại

Tôi chọn đúng ba feature, mỗi pool một feature: FR-02 Login và Account Lockout thuộc Pool A, FR-07 Shopping Cart thuộc Pool B, và FR-16 Product Import CSV thuộc Pool C. Bộ test có 45 test case duy nhất. Chạy trên ba browser tạo thành 135 lượt thực thi: 72 lượt pass, 63 lượt fail có chủ đích từ các test gắn tag @bug, và không có unexpected failure. Tổng cộng có chín browser run và chín HTML report riêng.

Chỉ tiếp vào bảng chi tiết:

FR-02 có 16 case, FR-07 có 17 case và FR-16 có 12 case, đều vượt ngưỡng tối thiểu 12 của đề. Ba engine Chromium, Firefox và WebKit cho kết quả nhất quán.

Mục 3 - Kiến trúc POM và Data-Driven Testing (1:50-3:10)

Thao tác

Mở lần lượt:

1. `tests/pages/CartPage.js`
2. `tests/data/fr07-cart-products.csv`
3. `tests/data/fr07-cart-cases.json`
4. `tests/fr07-cart.spec.js`

Tại spec, chỉ vào `readCsv`, `readJson` và vòng lặp tạo TC03-TC06.

Lời thoại

Bộ test áp dụng Page Object Model. CartPage chứa locator và hành động như thêm sản phẩm, mở giỏ bằng link header, lấy từng dòng sản phẩm. Spec giữ các assertion bám SRS. Dữ liệu không hardcode thành mảng trong spec. File CSV chứa tên sản phẩm, giá, quantity và expected subtotal; file JSON chứa message và các nhãn mong đợi. Spec đọc hai file này và dùng vòng lặp để sinh test, vì vậy đây là data-driven thật chứ không chỉ import file cho có.

Các assertion pattern gồm URL, visibility và text, element count, DOM attribute, exact money calculation, dialog event, API status/body và backend state. Như vậy vượt yêu cầu tối thiểu ba pattern.

Mục 4 - Human review: lỗi quan trọng đã sửa trong code AI sinh (3:10-4:35)

Thao tác

Trong `CartPage.js`, chỉ vào:

- `openFromHeader()`
- `addProductByName()`

Sau đó mở lịch sử hoặc Main Report tại R-02.

Lời thoại

Đây là một lỗi quan trọng trong bản AI sinh ban đầu. Sau khi thêm sản phẩm, AI dùng `page.goto('/cart')`. Tuy nhiên giỏ hàng của SUT chỉ nằm trong React Context. `page.goto` gây full reload, khởi tạo lại CartProvider và làm mất toàn bộ sản phẩm. Kết quả là nhiều test đỏ vì giỏ trống chứ không phải do FR-07 bị lỗi.

Tôi đã sửa bằng cách click link giỏ hàng trong header thông qua React Router. Đây là client-side navigation nên giữ nguyên state. Sau khi sửa, toàn bộ 11 case bình thường của FR-07 pass trên cả ba engine; chỉ sáu assertion bám SRS còn đỏ. Tôi cũng thay assertion yếu chỉ kiểm tra ký hiệu tiền bằng việc parse và so sánh exact subtotal và total từ CSV.

Mục 5 - Chạy live trên ba browser (4:35-6:25)

Thao tác

Chuyển terminal, chạy:

```
npm run test:multibrowser:fr07
```

Trong lúc chạy, không cần đọc từng dòng. Giải thích ngắn:

Lời thoại trong lúc chờ

Script này chạy cùng một spec lần lượt trên Chromium, Firefox và WebKit. Mỗi lần chạy ghi vào một report folder riêng. Playwright config tự khởi động ba service và metadata có Run by 23127259 cùng ISO timestamp.

Các test @bug được phép đỏ vì chúng assert theo SRS thay vì hành vi sai hiện tại của SUT. Runner phân biệt bugFailures với unexpectedFailures. Nếu selector sai, browser không khởi động hoặc một test bình thường fail thì unexpectedFailures sẽ khác không và command bị đánh dấu thất bại.

Khi bảng summary xuất hiện, phóng to và chỉ vào ba dòng.

Kết quả FR-07 trên mỗi browser là 11 pass, 6 bug failure, 0 unexpected failure. Ba engine cho cùng kết quả nên không còn flakiness hay browser-launch error.

Mục 6 - Mở HTML report và kiểm tra metadata (6:25-7:35)

Thao tác

Chạy:

```
npx playwright show-report playwright-report/fr07-cart-chromium
```

Trình duyệt mở report. Thực hiện:

1. Chỉ vào `Run by: 23127259`.
2. Chỉ vào ISO timestamp.
3. Mở một case xanh, ví dụ TC07 exact total.
4. Mở TC12 hoặc TC13 màu đỏ.
5. Chỉ vào Expected/Received và screenshot/trace.

Lời thoại

Đây là HTML report Chromium. Phần metadata hiển thị đúng Run by 23127259 và thời gian ISO, đáp ứng yêu cầu chống tạo evidence giả. Case xanh này kiểm tra exact total. Case TC12 được gắn @bug và kỳ vọng chỉ có một dòng quantity bằng hai khi thêm cùng sản phẩm hai lần. Thực tế SUT tạo hai dòng, nên assertion đỏ đúng lý do.

Mục 7 - Đối chiếu SRS, Bug Report và GitHub Issue (7:35-8:45)

Thao tác

1. Mở `eshop-sut/README.md`, chỉ FR-07: thêm cùng sản phẩm phải tăng quantity.
2. Mở `23127259/bug-report/BUG_REPORT.md`, BUG-10.
3. Chuyển Chrome sang Issue #25: <https://github.com/hungtmh/HW04-TESTING/issues/25>
4. Chỉ vào author `thangak18`, steps, expected, actual và evidence image.

Lời thoại

Tôi đối chiếu failure với SRS chứ không chỉ đọc source. FR-07 ghi rõ thêm cùng sản phẩm phải tăng số lượng và không tạo dòng mới. BUG-10 trong báo cáo có steps, expected, actual, severity, đề xuất sửa và evidence. GitHub Issue số 25 được tạo bằng tài khoản thangak18 và đính kèm cùng evidence. Toàn bài có 20 root defect tương ứng Issues số 16 đến 35.

Mục 8 - Git workflow và kết luận (8:45-9:35)

Thao tác

Mở PR #36:

<https://github.com/hungtmh/HW04-TESTING/pull/36>

Chỉ trạng thái `Merged`, author/merger `thangak18` và commit history.

Lời thoại

Mã nguồn được làm trên branch riêng, commit theo từng feature, sau đó mở và merge Pull Request số 36 bằng tài khoản thangak18. Repository hiện có ít nhất tám commit chạm file spec theo yêu cầu.

Tôi xin tổng kết: 45 test case, chín browser run, 72 pass, 63 bug failure có chủ đích, không có unexpected failure, 20 bug có evidence và GitHub Issue. Phần quan trọng nhất tôi học được là phải xác định đúng nguyên nhân test fail; không được làm yếu assertion chỉ để report chuyển sang màu xanh. Tôi cảm ơn thầy đã theo dõi.

PHẦN C - KIỂM TRA SAU KHI QUAY

- ☐ Video dài ít nhất 5 phút; khuyến nghị 8-10 phút.
- ☐ Âm thanh tiếng Việt nghe rõ.
- ☐ Có face-cam hoặc `whoami` + `hostname` rõ trên video.
- ☐ Thấy lệnh chạy đủ Chromium, Firefox, WebKit.
- ☐ Thấy bảng summary có `unexpectedFailures: 0`.
- ☐ Thấy HTML report có `Run by: 23127259` và ISO timestamp.

- ☐ Có giải thích ít nhất một lỗi do AI sinh và cách sửa.
- ☐ Có đối chiếu SRS -> failed assertion -> Bug Report -> GitHub Issue.
- ☐ Không lộ token, cookie, mật khẩu cá nhân.
- ☐ Upload YouTube ở chế độ **Unlisted**.
- ☐ Mở link bằng cửa sổ Incognito để kiểm tra người khác xem được.
- ☐ Gửi link để cập nhật README, Main Report và điểm tự đánh giá.

Phương án dự phòng

- Nếu live run quá lâu: vẫn phải quay ít nhất một feature chạy đủ ba browser; không dùng report cũ thay hoàn toàn cho thao tác chạy.
- Nếu port đang bị chiếm: đóng server cũ rồi chạy lại; không sửa `reuseExistingServer` trong lúc quay.
- Nếu report không tự mở: dùng lại lệnh `npm playwright show-report <report-folder>`.
- Nếu nói vấp: dừng 2 giây rồi nói lại từ đầu câu, sau đó cắt đoạn lỗi khi edit video.